

TREE 2

PROBLEM 9

QUESTION:

DS	Session	Tree 2	Question Information	Level 1 Challenge 79
Problem Description: You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet. You are required to answer Q queries of type u, c, where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries. Constraints $1 \leq N, Q \leq 10^5$ $1 \leq u, v \leq N$ c is a lowercase alphabet s_i is a lowercase alphabet for all $1 \leq i \leq N$ 1 is the root node Input format - First line: Two space-separated integers N and Q respectively - Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i) - Next $N - 1$ line: Two space-separated integers u and v denoting an edge between node u and node v - Next Q lines: An integer u and a space-separated character c Output format For each query, print the output in a new line. Note: It is guaranteed that the input generates a valid tree. Explanation for test case 1 Tree given in the sample input will look like that. <pre>graph TD; 1((1 a)) --- 2((2 b)); 1 --- 3((3 a));</pre> Number of nodes in the subtree of node 1 having 'a' stored in it is 2.				
▼ Logical Test Cases Test Case 1 INPUT (STDIN) <pre>3 1 aba 1 2 1 3 1 a</pre> EXPECTED OUTPUT <pre>2</pre> Test Case 2 INPUT (STDIN) <pre>4 1 wsx 1 2 1 3 1 1 1 w</pre> EXPECTED OUTPUT <pre>10</pre>				
▼ Mandatory Test Cases Test Case 1 KEYWORD <pre>while(Q--)</pre> Test Case 2 KEYWORD <pre>for(i = 0; i < N - 1; i++)</pre>				

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
4	330	49

SOURCE CODE:

```
#include <stdio.h>

#define MAX_V 100005

int head_list[MAX_V], edge_to[2 * MAX_V], next_edge_ptr[2 * MAX_V], total_edges = 0;
char vertex_char[MAX_V];
int char_histogram[MAX_V][26];

void add_tree_edge(int u, int v) {
    edge_to[++total_edges] = v;
    next_edge_ptr[total_edges] = head_list[u];
    head_list[u] = total_edges;

    edge_to[++total_edges] = u;
    next_edge_ptr[total_edges] = head_list[v];
    head_list[v] = total_edges;
}

void traverse_subtree(int u, int parent) {
    char_histogram[u][vertex_char[u] - 'a'] = 1;
    for (int e = head_list[u]; e; e = next_edge_ptr[e]) {
        int neighbor = edge_to[e];
        if (neighbor != parent) {
            traverse_subtree(neighbor, u);
            for (int ch = 0; ch < 26; ch++) {
                char_histogram[u][ch] += char_histogram[neighbor][ch];
            }
        }
    }
}

int main() {
    int node_count, query_count;
    if (scanf("%d %d", &node_count, &query_count) != 2) return 0;

    char word[MAX_V];
    scanf("%s", word);
    for (int i = 1; i <= node_count; i++) {
        vertex_char[i] = word[i - 1];
    }

    for (int i = 0; i < node_count - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_tree_edge(u, v);
    }

    traverse_subtree(1, 0);

    while (query_count--) {
        int target_node;
        char target_char;
        scanf("%d %c", &target_node, &target_char);
        printf("%d\n", char_histogram[target_node][target_char - 'a']);
    }
    return 0;
}
```

OUTPUT:

```
3 1
aba
1 2
1 3
1 a
2
```

```
4 1
wsx
1 2
1 3
1 1
1 w
10
```